M. C. E Society's
Allana Institute of Management Sciences, Pune

Synopsis
on

Cine Circuit

Online Movie Ticket Booking System

Mini Project

By
Faizan Khan

MCA – I
Roll No. --

Academic Year 2025-26

Guide Name
Mehdi Gafri

ABSTRACT

The **Cine Circuit** is a production-grade, full-stack web-based Online Movie Ticket Booking System designed to provide a seamless platform for users to browse movies, view showtimes, select seats, and book tickets online. The system serves four distinct user roles — **Viewers** (general users), **Organizers** (movie creators), **Theatrers** (theatre owners), and **Administrators** — each with dedicated dashboards and role-specific functionalities.

The platform eliminates physical box-office queues by enabling online seat selection and payment through an integrated **Razorpay** payment gateway. Secure authentication uses JWT in httpOnly cookies and bcrypt password hashing. Automated HTML email notifications are sent for OTP verification, booking confirmation, cancellations, password resets, organizer approvals, and maintenance alerts — 17 distinct email templates in total.

Cine Circuit is built with **React.js 19 (Vite)** and **Tailwind CSS** on the frontend, **Node.js + Express.js** on the backend, and **MongoDB Atlas** as the cloud database. Media is managed via **Cloudinary**, and the app is deployed on **Vercel** (frontend) and **Render** (backend). Advanced capabilities include an AI-powered movie recommendation engine, maintenance mode with user notifications, bug reporting with media attachments, SEO optimization with JSON-LD schemas, Progressive Web App (PWA) support, automated cron jobs for movie status updates, visitor tracking, Swagger API documentation, and a comprehensive analytics dashboard.

Introduction

Cine Circuit is a comprehensive full-stack web application that automates and digitizes the movie ticket booking process. Traditional systems require physical visits to a box office, which is time-consuming and inconvenient. Cine Circuit solves this by providing an intuitive platform where users can register, browse movies, select seats interactively, make payments, and receive tickets — all online.

The system supports four roles: Viewers browse and book tickets; Organizers create and manage shows, upload posters/trailers, manage cast and hashtags, and submit verification documents; Theatrers manage venue details, allotted shows, ticket distribution, and sales analytics; Administrators oversee the entire platform — verifying shows/organizers/theatres, managing genres and languages, viewing analytics, and controlling maintenance mode.

Objectives of the Project

The primary objective is to design and develop a **full-stack, production-grade Online Movie Ticket Booking System** that covers the entire lifecycle from movie creation and show scheduling to seat selection, payment, ticket delivery, and post-show reviews.

Specific Objectives:

1. Provide a multi-role online platform (Viewer, Organizer, Theatrer, Admin) with JWT-based secure authentication.
2. Enable OTP-based email verification, password reset, and rate-limited profile updates (30-day cooldowns).
3. Integrate Razorpay payment gateway with atomic seat locking to prevent double-booking race conditions.
4. Allow Organizers to create shows with posters, trailers, cast, hashtags, genre, language, and ticket categories.
5. Allow Theatrers to manage allotted shows, distribute tickets, view sales analytics, and update show timings.
6. Provide Admins with full platform control: verify shows/organizers/theatres, manage genres/languages, and view dashboards.
7. Deliver 17 automated HTML email notifications covering all major platform events.
8. Implement an AI-powered movie recommendation engine using genre and sub-genre preferences.
9. Support SEO with react-helmet-async, JSON-LD schemas, dynamic sitemap.xml, and robots.txt.
10. Provide PWA support visitor tracking swagger api docs automated cron jobs for movie status updated

Modules

Module 1: User Authentication & Account Management

Handles registration, OTP email verification, login, logout, password reset, and profile updates (picture, username, phone, password) with rate-limiting cooldowns for all four user roles.

Module 2: Movie & Show Management

Organizers create shows with full metadata — title, tagline, genre, sub-genre, language, release date, cast, director, producer, writers, budget, poster, trailer, and hashtags. Show status is auto-managed (Upcoming → Released → Expired) by cron jobs.

Module 3: Ticket Booking & Seat Selection

Viewers browse shows, pick a theatre and date, select ticket count per category (Standard/Premium/VIP/Family/Loyalty), and proceed to checkout with live availability checks.

Module 4: Payment & Order Processing

Razorpay order creation with a pre-payment DB check, atomic findOneAndUpdate on verification, QR-code ticket generation, email confirmation, and cancellation with seat release.

Module 5: Theatre Management

Theatrers register their venue, get admin approval, receive show allotments, distribute tickets to buyers, view per-show revenue, and update show timings — with email notifications to affected ticket holders.

Module 6: Organizer Verification & Profiles

Organizers submit comprehensive professional profiles (role, experience, projects, awards, social media) and role-specific forms (Director/Producer × Fresher/Experienced).
Admin reviews with approval/rejection and up to 3 attempts before lockout.

Module 7: Admin Dashboard & Control Panel

Admins manage users, verify organizers/theatres/shows, perform CRUD on genres, sub-genres and languages, delete comments, view analytics charts, toggle maintenance mode, and handle bug reports.

Module 8: AI Movie Recommendation Engine

A public genre-based recommendation system: user selects genre → sub-genres shown → user selects sub-genre → top-8 most-liked matching movies returned.

Module 9: Email Notification System

17 HTML email templates auto-sent for: OTP, booking confirmation, payment failure, ticket PDF, password reset, profile updates (name/phone/image/password), organizer approval/rejection, theatre approval/rejection, maintenance alert, bug report submission/resolution.

Module 10: Bug Reporting System

Logged-in users submit bug reports (title, description, up to 5 images + 2 videos). Auto-generates a unique BUG-XXXXXX ID. Admins filter by status, update with notes, and trigger resolution emails.

Module 11: Rating, Review & Engagement

Viewers post star ratings (1-5) and text reviews (purchase required), post/delete comments (max 500 chars), like/dislike movies, add to wishlist, and receive a review-prompt popup 3 hours after a show ends.

Module 12: SEO, PWA & Performance

React Helmet meta tags + JSON-LD Movie schema per movie page, dynamic sitemap.xml endpoint, robots.txt, PWA manifest + service worker, lazy-loaded routes, cookie consent banner, visitor tracking, and mobile-blocker for screens < 768px.

Scope and Functionality of Each Module

Authentication & Account Management

- OTP auto-expires in 2 minutes; resend cooldown enforced
- JWT stored in httpOnly cookie; bcrypt password hashing (10 salt rounds)
- Role-based access: Viewer / Organizer / Theatrer / Administrator
- Update cooldowns: profile image & username & password (30 days), phone (7 days)
- Last login timestamp array tracked per user

Movie & Show Management

- Show metadata: title, tagline, genre, sub-genre, language, cast, hashtags, budget
- Poster (max 100MB) and trailer upload to Cloudinary
- Cron job runs every 6 hours: Upcoming → Released → Expired auto-transition
- Admin verification gate before show goes live on platform
- Custom messages per show for announcements

Ticket Booking & Payment

- Ticket categories: Standard, Premium, VIP, Family, Loyalty
- Fresh DB re-check before Razorpay order creation (prevents overselling)
- Atomic findOneAndUpdate on payment verification (prevents race conditions)
- QR code embedded in ticket; PDF generated and emailed to user
- Cancellation workflow: seat released, refund initiated, email sent

Theatre & Organizer Management

- Theatre registration → Admin approval (Pending/Approved/Rejected workflow)
- Show allotment system: Organizer allots theatre → Theatre sees it in dashboard
- Organizer profile: up to 3 verification attempts; lockout after 3 failures
- Director/Producer × Fresher/Experienced sub-forms with word-limited fields
- Theatre sales analytics: revenue per show, payment status, unsold tickets

Admin Panel & Bug Reports

- CRUD operations on Genre, Sub-Genre, Language
- User lists: verified/unverified users, organizers, and theatres
- Analytics dashboard with booking and revenue charts
- Maintenance mode: custom message, end time, optional mass email to all users
- Bug reports: paginated list, status filter, media preview, admin notes

Technology Used

A **System Requirement Specification (SRS)** outlines the essential hardware and software requirements for the development, deployment, and operation of Cine Circuit.

Hardware Requirements

- Processor: Intel Core i3 / AMD Ryzen 3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 100 GB (SSD preferred)
- Internet Connection: Required (cloud-based system)

Software Requirements

- Operating System: Windows 10/11 or Linux (Ubuntu)
- Runtime: Node.js v20.x
- Frontend: React.js v19 + Vite 6 + Tailwind CSS v4 + Redux Toolkit
- Backend: Node.js + Express.js v4
- Database: MongoDB (MongoDB Atlas cloud) via Mongoose v8
- Authentication: JSON Web Tokens (JWT) + bcrypt (httpOnly cookies)
- Payment: Razorpay SDK
- Media Storage: Cloudinary (images, videos, PDFs)
- Email: Nodemailer (SMTP)
- File Upload: express-fileupload (10MB images / 100MB video)
- Rate Limiting: express-rate-limit (5 auth attempts / 200 general per 15 min)
- Scheduled Jobs: node-cron (every 6 hours for movie status updates)
- Visitor Tracking: express-visitor-counter
- API Documentation: Swagger UI (swagger-jsdoc + swagger-ui-express)
- SEO: react-helmet-async + JSON-LD + dynamic sitemap.xml
- PWA: Web App Manifest + Service Worker
- Animations: GSAP + Swiper.js
- Forms: React Hook Form
- Deployment: Vercel (frontend) + Render (backend)
- Version Control: Git + GitHub
- Browser: Google Chrome / Mozilla Firefox

System Diagrams

Use Case Diagram

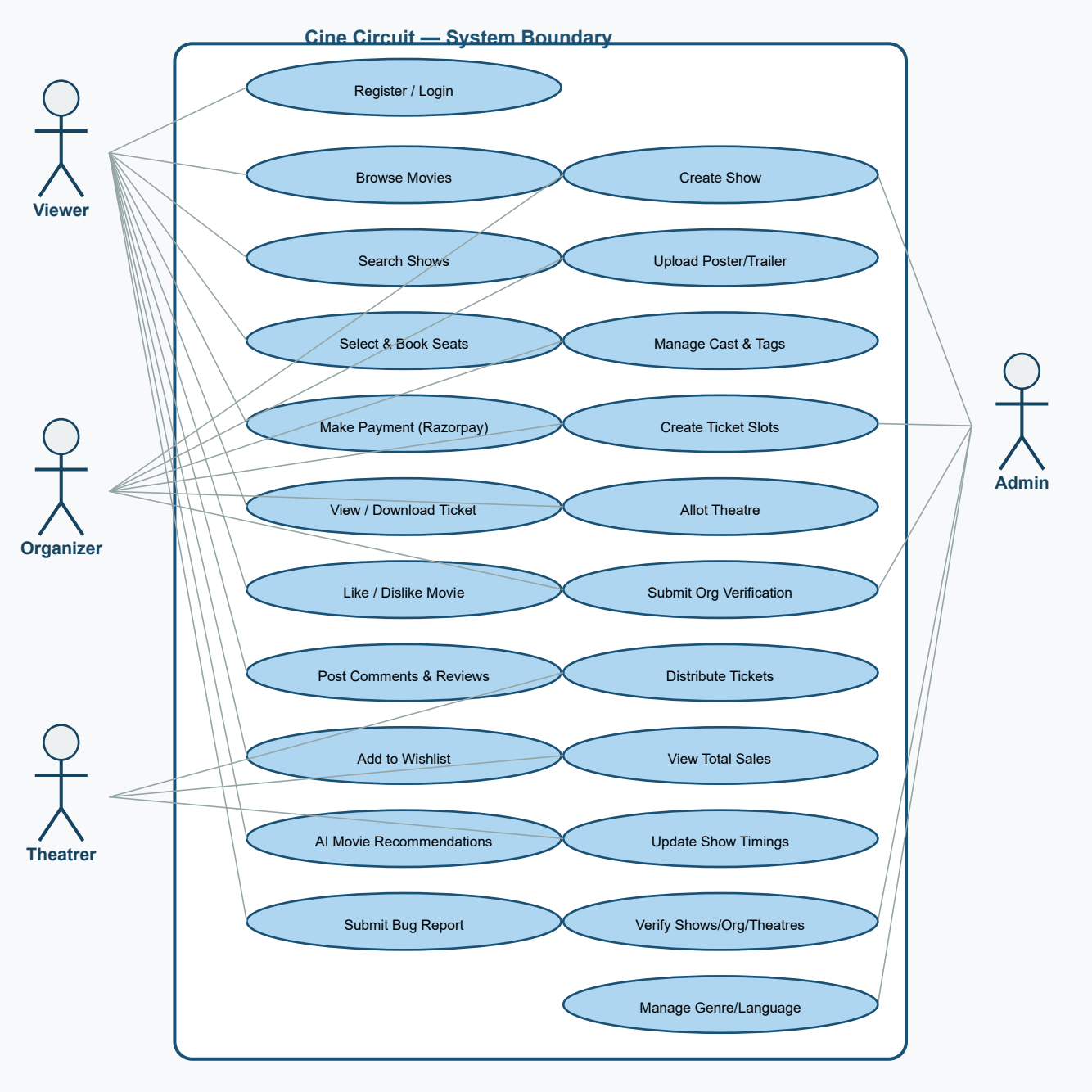

Figure 1: Use Case Diagram — Cine Circuit (4 Actors)

Activity Diagram — Ticket Booking Flow

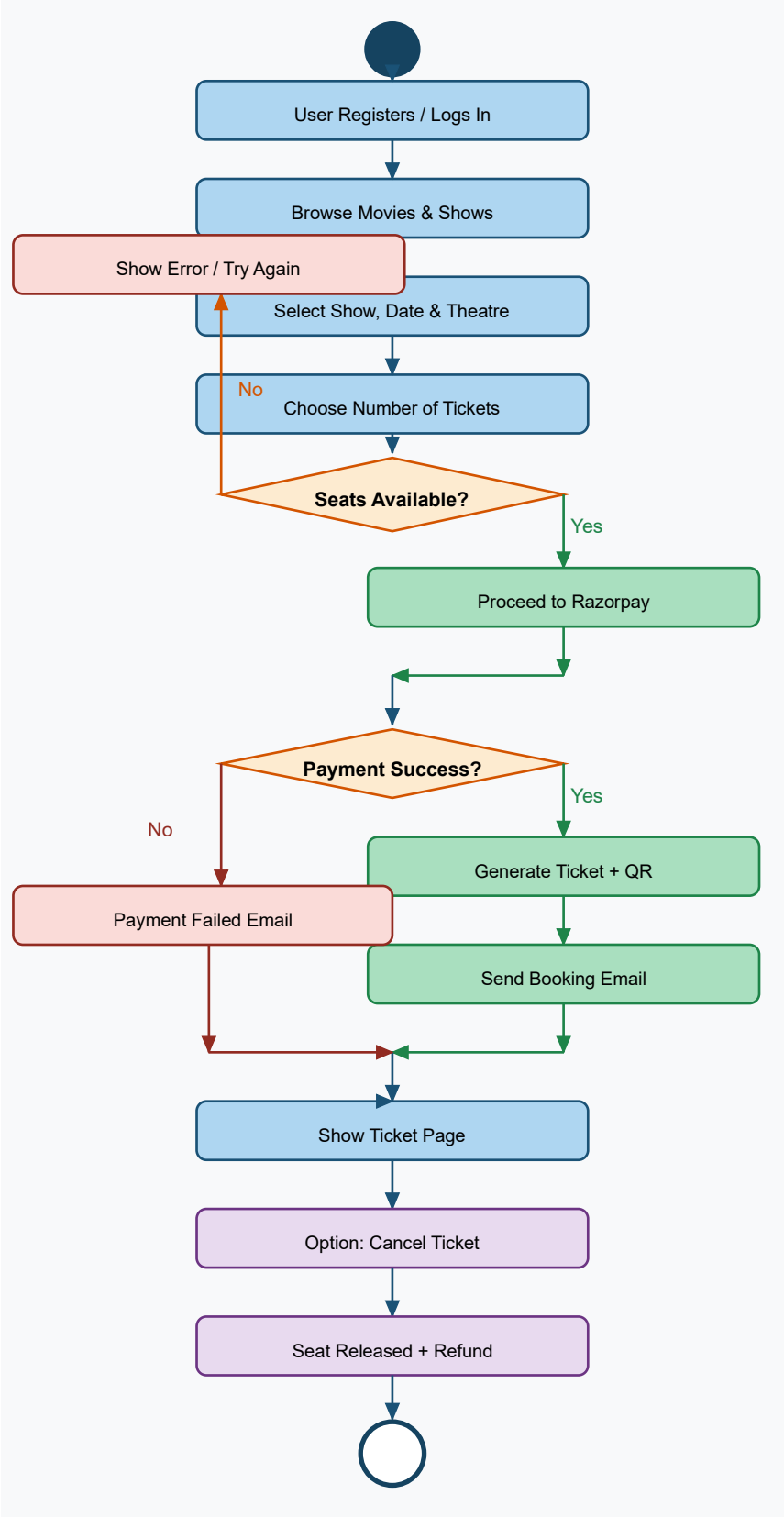

Figure 2: Activity Diagram — Full Ticket Booking & Cancellation Process

Sequence Diagram — Booking Interaction

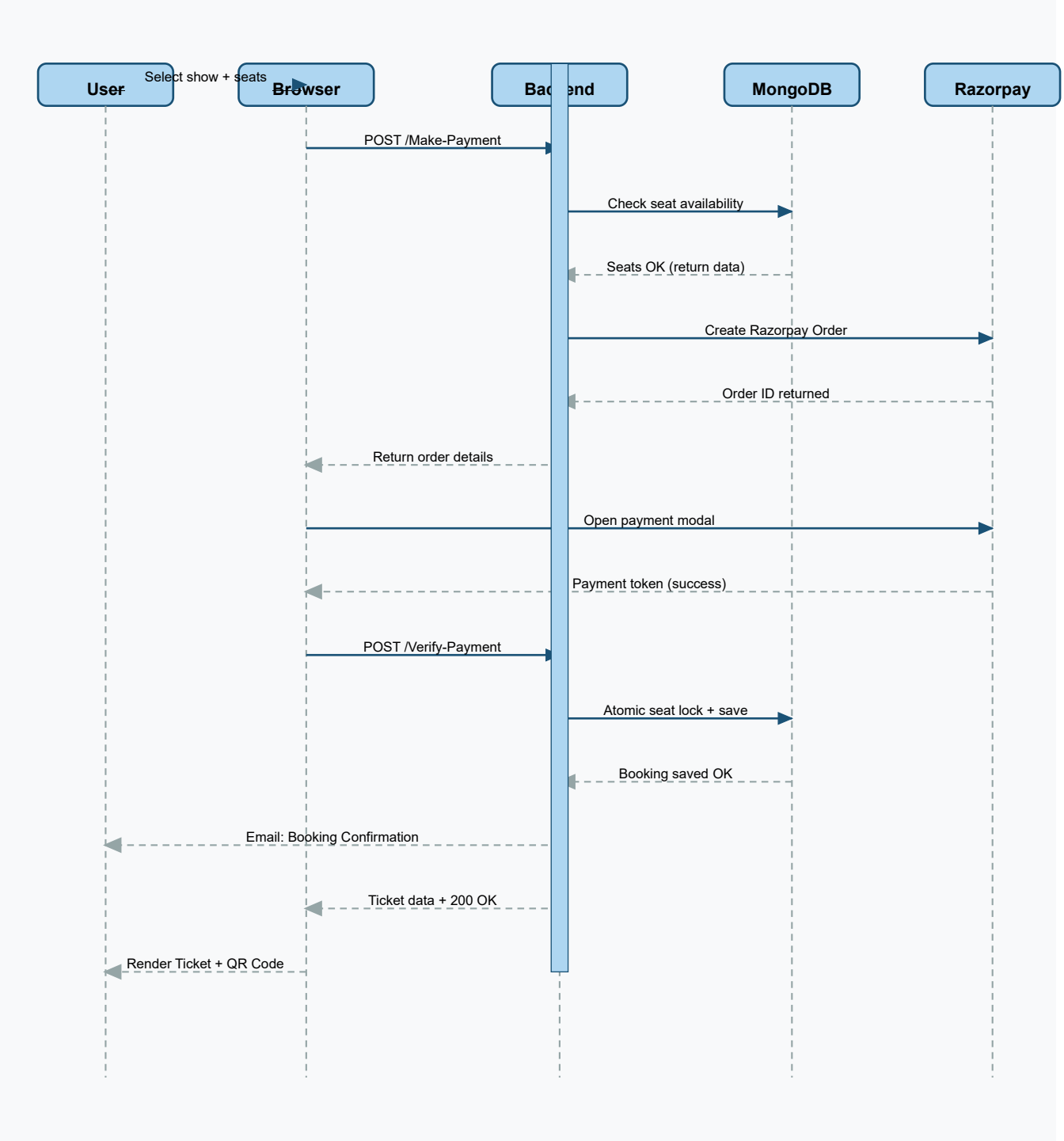

Figure 3: Sequence Diagram — User → Browser → Backend → MongoDB → Razorpay

Database / ER Diagram

Figure 4: ER Diagram — 16 MongoDB Collections with Relationships

Limitations of the Proposed System

1 No native mobile app (web-based only; PWA partial offline support included).

2 Seat selection is count-based, not individual interactive seat map.

3 AI recommendation uses genre-based filtering only, not collaborative filtering.

4 Seat availability uses DB-level atomic operations, not WebSocket real-time sync.

5 Refund initiation is manual; automatic bank refund not implemented.

6 JWT stored in localStorage on client side (XSS risk — BUG-2 noted for future fix).

7 UI is English-only; no multi-language interface support.

8 Chat module is a placeholder; real-time messaging not yet implemented.

Conclusion

Cine Circuit successfully demonstrates the design and development of a full-stack, production-grade Online Movie Ticket Booking System. By integrating modern technologies — React.js, Node.js, MongoDB Atlas, Razorpay, and Cloudinary — the project delivers a real-world applicable solution covering the complete lifecycle from movie creation to ticket delivery.

The platform supports four user roles, 79 REST API endpoints, 26 database models, 17 automated email templates, and 12 major feature modules. Advanced features like AI movie recommendations, automated cron jobs, maintenance mode, SEO with JSON-LD schemas, PWA support, visitor tracking, and a Swagger-documented API make Cine Circuit a comprehensive, scalable, and production-ready system.

This project provided valuable hands-on experience in full-stack web development, RESTful API design, database modeling with MongoDB, cloud deployment, secure authentication, payment gateway integration, and software engineering best practices — reflecting practical implementation of academic concepts and readiness for industry-level development.

Student Name : Faizan Khan

Student Mobile Number : 9011575978

E-mail Id : FaizanKhan901152@gmail.com

Permanent Address : lane no 32 near alfall masjid bhagyoday nagar pune